python基础面试题

1.解释 if name == "main": 的作用

当文件被直接运行才会执行下面的代码，被 import 时不会。

2.列表推导式与生成器表达式的区别

列表推导式一次性生成完整列表；

生成器表达式惰性迭代，节省内存。

3.解释 *args 和 **kwargs

*args 接收多余位置参数形成元组；**kwargs 接收多余关键字参数形成字典。

4.解释 new 和 init 的区别?

`__new__` 负责创建实例并返回，先于 `__init__` 执行；`__init__` 负责初始化已创建好的实例。

5.Python 中的装饰器本质是什么?

装饰器本质是高阶函数，它接收一个函数并返回一个新函数，运行时通过 @ 语法糖应用，常用于日志、权限、缓存等横切逻辑。

6.如何防止列表在遍历时被修改?

遍历时使用切片拷贝：for x in lst[:]，或用 list(lst) 创建副本后再遍历。

7.with 语句的上下文管理器原理?

对象实现 `__enter__` 和 `__exit__` 协议。with 块进入时调用 `__enter__`，退出时 `__exit__` 负责清理资源（如关闭文件、释放锁）。

8.== 和 is 有什么区别?

== 比较值是否相等；is 比较对象身份（内存地址）是否相同。类似 id(a) == id(b)。

9.解释生成器（generator）和迭代器（iterator）的区别?

迭代器是实现了 `__iter__` + `__next__` 的任何对象；生成器是用 yield 关键字的函数，自动实现迭代器协议，写法更简洁。

10.怎样实现自定义对象的排序?

实现 **lt** 或在 sorted() 里传 key= 函数。例如 sorted(users, key=lambda u: u.age)。

11.列表和元组区别？什么时候用元组?

列表可变、元组不可变；元组能当字典键、集合的元素，占内存更小；函数参数解包、常量数据用元组。

12.解释 all 在模块里的作用?

是一个列表形式，定义 from module import * 时哪些名字被导出；未列入的不会导入，可用于隐藏内部实现。

13.解释 map、filter、reduce 的用法差异。

map(func, iterable) → 对每个元素调用函数，返回迭代器；

filter(func, iterable) → 保留返回 True 的元素；

reduce(func, iterable) → 连续把函数结果与下一个元素累计，得单一值（需 from functools import reduce）。

14.break、continue、pass 各有什么作用?

break 跳出整个循环；

continue 跳过本次迭代；

pass 占位，啥也不干。

15.del obj 真的立刻释放内存吗?

不一定；del 只是减少引用计数，引用计数为 0 时由解释器择机回收，循环引用要等 GC 触发。

16.range() 返回的是列表吗？占用内存会随范围变大而变大吗?

不是，是惰性迭代器 range 对象；无论范围多大，它只保存 start/stop/step，占用固定内存。

17.try...finally 中如果 return，finally 还会执行吗?

会，finally 的代码一定执行，并且其内部的 return 会覆盖前面的 return。

18.解释 str.join() 为什么比循环 += 拼接字符串快?

+= 每次创建新字符串，时间复杂度 $O(n^2)$；str.join() 预分配总长度，只拷贝一次，时间复杂度 O(n)。

19.解释 import module 与 from module import name 在命名空间上的区别。

前者把模块对象绑定到当前命名空间，需 module.name 访问；后者把 name 直接拉进来，后续覆盖同名变量不会影响原模块。

20.list.sort() 的稳定性是什么意思？如何利用这一点?

稳定性指相等元素的相对顺序保持不变，可用于多级排序：先按次要键排序，再按主要键排序，无需额外字段。

例如：先按年龄排序，如果年龄相同则按分数排序

从最终结果来看，排序优先级是：

1. 第一优先级（主要）：分数（当年龄相同时，用分数来决定顺序）
2. 第二优先级（次要）：年龄（首先按这个分组）

21.内存只有4G，如何读取一个大小为8G的文件?

在Python中读取文件可以先通过open函数获取文件对象，在读取文件时，可以通过read方法的size参数指定读取的大小，也可以通过seek方法的offset参数指定读取的位置，这样就可以控制单次读取数据的字节数和总字节数。